R Cheat Sheet

Christian Knudsen

pdf

Det er ikke meget R I faktisk skal bruge.

Og egentlig ville jeg foreslå at I deltog på et af vores introkurser til R.

Dette er langt fra færdigt. Fortæl gerne hvad der er brug for!

Gem en værdi

Det er lettere at skrive "SE" end at skrive 0.4847285

```
SE <- 0.4847285
```

Og hvis vi har en beregning af SE, kan vi lige så godt gemme resultatet direkte:

```
SE <- 3.1414/sqrt(42)
```

Vil vi gerne se værdien igen, skriver vi "bare" `SE`

```
SE
```

```
[1] 0.4847285
```

HUSK der er forskel på store og små bogstaver. `SE` er noget andet end `se`.

Vektorer

Vi har undertiden behov for at arbejde med mere end en værdi. Det kalder vi en "vektor"

Vi laver vektorer med funktionen `c()`:

```
vektor <- c(42, 47, 666)
```

Hvis vi vil se den igen:

```
vektor
```

```
[1]  42  47 666
```

R er hvad vi kalder vektoriseret. De fleste (men ikke alle!) operationer vil virke på hvert enkelt element i vektoren:

```
vektor + 1
```

```
[1]  43  48 667
```

Bemærk at vi lægger 1 til hver værdi i vektoren.

Det er derfor vi i stedet for at skrive

```
10 + 1.96*SE
```

```
[1] 10.95007
```

og

```
10 - 1.96*SE
```

```
[1] 9.049932
```

for at få et konfidensinterval, kan skrive:

```
10 + c(-1,1)*1.96*SE
```

```
[1]  9.049932 10.950068
```

`c(-1,1)` ganget med 1.96 giver

```
c(-1,1)*1.96
```

```
[1] -1.96  1.96
```

Ganger vi også med SE får vi:

```
c(-1,1)*1.96*SE
```

```
[1] -0.9500679  0.9500679
```

Og lægger vi 10 til det, lægger vi 10 til hvert element i det resultat.

Bemærk også at vi ikke ændrede på vektoren:

```
vektor
```

```
[1]  42  47 666
```

Vil vi gøre det skal vi huske at gemme resultatet i vektoren. Eller "objektet" som vi også kalder den_

```
vektor <- vektor + 1
```

Nu er `vektor` ændret:

```
vektor
```

```
[1]  43  48 667
```

Funktioner virker også på vektorer – på hvert element:

```
sqrt(vektor)
```

```
[1]  6.557439  6.928203 25.826343
```

Hvad var en funktion?

Det er en "opskrift" eller metode der tager noget input, og giver noget output.

Vi så `sqrt()`, der giver os kvadratroden på inputtet. Da vi gav den en vektor, gav den os kvadratroden på hvert element eller værdi i vektoren.

Inden vi går videre med funktioner, ser vi lige på manglende værdier.

Dem får I næppe brug for til eksamen, men det er nyttigt at kende alligevel.

Nu laver vi en ny vektor der indeholder værdierne fra den gamle vektor, og en manglende værdi:

```
ny_vektor <- c(vektor, NA)
```

Manglende værdier angiver vi med "NA". Husk at "NA" er noget andet end "na" eller "Na".

Læg mærke til at når vi skriver `vektor`, spytter R værdierne i `vektor` ud. Så `c()` funktionen får værdierne fra den oprindelige vektor, og en ekstra værdi, `NA`.

Manglende værdier optræder ofte i den virkelige verden. De angiver at "her skulle der have været en værdi, men vi ved ikke hvad den er".

Tilbage til funktioner.

`mean()` funktionen giver middelværdien eller gennemsnittet af inputtet:

```
mean(vektor)
```

```
[1] 252.6667
```

Men hvis der er en manglende værdi i inputtet, får vi at vide at resultatet er ukendt:

```
mean(ny_vektor)
```

```
[1] NA
```

Det gør vi fordi middelværdien findes ved at lægge alle tallene sammen, og dividere med antallet. Og resultatet af 42 + NA er ukendt. Vi ved jo ikke hvad den ukendte værdi dækker over. Derfor er resultatet af `mean()` også ukendt hvis inputtet indeholder ukendte værdier.

Funktioner kan tage mere end et input. Det så vi i `c()` funktionen, der kunne tage mange inputs. Hvert input adskilte vi med et komma.

Funktioner kan også tage en speciel type input, som vi kalder argumenter. Det er input der ændrer på den måde funktionen fungerer.

De har et navn, og skal have en værdi. `mean()` har et specielt argument `na.rm`. Hvis `na.rm` er sand, fjerner `mean()` de manglende værdier før middelværdien beregnes:

```
mean(ny_vektor, na.rm = TRUE)
```

```
[1] 252.6667
```

Husk at "TRUE" er noget andet end "true".

Langt de fleste funktioner har argumenter. Nogen af dem *skal* vi angive:

```
mean(x = ny_vektor)
```

```
[1] NA
```

Og hvis vi angiver dem i den rækkefølge funktionen forventer at få dem i, behøver vi ikke skrive navnet på dem. Derfor virker

```
mean(ny_vektor)
```

```
[1] NA
```

uden at vi angiver at argumentet hedder "x".

Bortset fra det første argument, som vi sjældent gider at navngive, er det en rigtig god ide at angive navnene på de øvrige argumenter.

Så behøver man nemlig ikke selv huske rækkefølgen.

Argumenter har normalt en "default" værdi. Den værdi funktionen antager at argumentet har, hvis vi ikke fortæller andet.

`mean()` kan tage argumentet `na.rm`. Men hvis vi ikke angiver noget, antager `mean()` at værdien som default er `FALSE`. Vil vi have funktionen til at gøre noget andet end default, skal vi fortælle den det.

Listen over argumenter finder vi ved at køre kommandoen:

```
?mean
```

Det åbner hjælpefilen, og der kan vi læse hvilke argumenter funktionen kan tage.

Nyttige funktioner:

`c()`

`sqrt()`

`mean()`

`median()`

`sd()`

`qnorm`

`pnorm`